

DISTRIBUTED GENOMIC VARIANT ANALYSIS USING APACHE SPARK AND GLNEXUS

CODE:

1) Preprocess.py

```
import os
```

```
import subprocess
```

```
import time
```

```
import glob
```

```
from pyspark.sql import SparkSession
```

```
def main():
```

```
    spark = SparkSession.builder \
```

```
        .appName("Genomic_Preprocessing") \
```

```
        .master("local[*]") \
```

```
        .getOrCreate()
```

```
    project_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
```

```
    input_dir = os.path.join(project_dir, "data", "input_bams")
```

```
    output_dir = os.path.join(project_dir, "data", "output_bams")
```

```
    # Find the GATK jar directly instead of using the wrapper script
```

```
    gatk_jar = glob.glob(os.path.join(project_dir, "tools", "gatk-4.5.0.0", "gatk-package-  
*.jar"))[0]
```

```
    print(f"Using GATK jar {gatk_jar}")
```

```
    bam_files = [f for f in os.listdir(input_dir) if f.endswith(".bam")]
```

```

print(f"\nFound {len(bam_files)} samples to process.")

for bam in bam_files:

    sample_name = bam.split(".")[0]

    input_path = os.path.join(input_dir, bam)

    output_path = os.path.join(output_dir, f"{sample_name}_dedup.bam")

    metrics_path = os.path.join(output_dir, f"{sample_name}_metrics.txt")


    print(f"\n--- Starting Preprocessing for {sample_name} ---")

    start_time = time.time()


    # Call Java directly — subprocess list handles spaces in paths safely

    cmd = [

        "java",

        "-Dsamjdk.use_async_io_read_samtools=false",

        "-Dsamjdk.use_async_io_write_samtools=true",

        "-Dsamjdk.use_async_io_write_tribble=false",

        "-Dsamjdk.compression_level=2",

        "-jar", gatk_jar,

        "MarkDuplicatesSpark",

        "-I", input_path,

        "-O", output_path,

        "-M", metrics_path,

        "--conf", "spark.executor.cores=4"

    ]


    try:

        subprocess.run(cmd, check=True)

```

```

        elapsed = time.time() - start_time

        print(f"[SUCCESS] {sample_name} preprocessed in {elapsed:.2f} seconds.")

    except subprocess.CalledProcessError as e:

        print(f"[ERROR] Failed to process {sample_name}. Check the logs.")

spark.stop()

if __name__ == "__main__":
    main()

```

2) variant_calling.py

```

import os
import subprocess
import time

def main():
    project_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
    ref_path = os.path.join(project_dir, "data", "reference", "20.fa")
    input_dir = os.path.join(project_dir, "data", "output_bams")
    output_dir = os.path.join(project_dir, "data", "results")
    gatk_path = os.path.join(project_dir, "tools", "gatk-4.5.0.0", "gatk")

    # Create results folder if it doesn't exist
    os.makedirs(output_dir, exist_ok=True)

    # Find the cleaned BAM files
    dedup_files = [f for f in os.listdir(input_dir) if f.endswith("_dedup.bam")]

```

```
print(f"Starting Variant Calling for {len(dedup_files)} samples...")
```

```
for bam in dedup_files:
```

```
    sample_name = bam.replace("_dedup.bam", "")
```

```
    input_path = os.path.join(input_dir, bam)
```

```
    output_vcf = os.path.join(output_dir, f"{sample_name}.vcf")
```

```
    print(f"\n>>> Calling variants for: {sample_name}")
```

```
    start_time = time.time()
```

```
    # GATK command for HaplotypeCaller
```

```
    cmd = [
```

```
        "python3", gatk_path, "HaplotypeCaller",
```

```
        "-R", ref_path,
```

```
        "-I", input_path,
```

```
        "-O", output_vcf,
```

```
        "-L", "20:10000000-10200000" # Focus on the region we downloaded
```

```
    ]
```

```
    try:
```

```
        subprocess.run(cmd, check=True)
```

```
        elapsed = time.time() - start_time
```

```
        print(f"[SUCCESS] Created {sample_name}.vcf in {elapsed:.2f} seconds.")
```

```
    except subprocess.CalledProcessError as e:
```

```
        print(f"[ERROR] Variant calling failed for {sample_name}.")
```

```
if __name__ == "__main__":
```

```
    main()
```

3) analyse_dna.py

```
from pyspark.sql import SparkSession
```

```
import pyspark.sql.functions as F
```

```
import pandas as pd
```

```
print("🚀 Booting up Spark Engine for Deep Analysis...")
```

```
spark = SparkSession.builder \
```

```
    .appName("Genomes_Analytics") \
```

```
    .master("local[*]") \
```

```
    .getOrCreate()
```

```
print("📖 Reading Parquet data from HDFS...")
```

```
df =
```

```
spark.read.parquet("hdfs://localhost:9000/genomics_project/parquet_data/chr22.adam")
```

```
print("⚙️ Crunching advanced genomic data...")
```

1. Total Count & Basic Stats (Keep this for the top of the dashboard)

```
total_mutations = df.count()
```

```
common_variants =
```

```
df.groupBy("referenceAllele").count().orderBy(F.desc("count")).limit(5).toPandas()
```

```
common_variants['total_mutations_analyzed'] = total_mutations
```

```
common_variants.to_csv("spark_results.csv", index=False)
```

2. NEW: Extract exact mutation locations and DNA swaps

We pull the Chromosome (contigName), the exact position (start), and what the DNA mutated into

```

print("🧬 Extracting specific chromosomal anomaly locations...")

locations_df = df.select(
    F.col("referenceName").alias("Chromosome"),
    F.col("start").alias("Position"),
    F.col("referenceAllele").alias("Original_DNA"),
    F.col("alternateAllele").alias("Mutated_DNA")
).limit(100) # Grabbing a sample of 100 specific anomalies

# Save this detailed location data for the web dashboard
locations_df.toPandas().to_csv("mutation_locations.csv", index=False)

print("✅ SUCCESS! Advanced data extracted.")

spark.stop()

```

4) app.py

```

import streamlit as st
import pandas as pd
import requests
import time

st.set_page_config(page_title="Genomics Disease Predictor", layout="wide")

st.title("🧬 Pediatric Genomic Disease Predictor")

st.markdown("**Live Clinical Database Connection:** Ping `MyVariant.info` REST API")
st.divider()

# --- THE LIVE API FUNCTION ---

def fetch_disease_data(chrom, pos, ref, alt):

```

```

"""Pings the live medical database to find diseases linked to a mutation."""

# Format the exact DNA change (HGVS standard format)
hgvs_id = f"chr{chrom}:g.{pos}{ref}>{alt}"

url = f"https://myvariant.info/v1/variant/{hgvs_id}?fields=clinvar"

try:
    response = requests.get(url, timeout=3)

    if response.status_code == 200:
        data = response.json()

        # Dig into the clinical database response to find the disease name
        if 'clinvar' in data:
            rcv = data['clinvar'].get('rcv', {})

            # APIs can return lists or dictionaries, so we handle both safely
            if isinstance(rcv, list):
                rcv = rcv[0]

            disease = rcv.get('conditions', {}).get('name', 'No associated disease
documented')

            risk = rcv.get('clinical_significance', 'Unknown')

            return disease, risk
        except Exception as e:
            pass

    return "Benign / No Disease Found", "Low Risk"

# --- MAIN DASHBOARD ---

try:
    # Load the raw mutation locations your Spark engine found

```

```
location_data = pd.read_csv("mutation_locations.csv")
```

```
col1, col2 = st.columns([1, 2])
```

```
with col1:
```

```
    st.subheader("Child's Raw Genetic Anomalies")
```

```
    st.markdown("These are the exact chromosomal mutations extracted by the Spark  
Hadoop engine.")
```

```
    st.dataframe(location_data.head(15), use_container_width=True) # Show top 15 for  
speed
```

```
with col2:
```

```
    st.subheader(" Live Clinical Disease Scan")
```

```
    st.markdown("Cross-reference the child's mutations with global disease  
databases.")
```

```
if st.button("Run Live API Disease Prediction"):
```

```
    # Create an empty box to show live progress
```

```
    progress_text = "Establishing secure connection to Clinical API..."
```

```
    my_bar = st.progress(0, text=progress_text)
```

```
    results = []
```

```
    # Scan the top 10 most critical mutations to keep the API fast
```

```
    top_mutations = location_data.head(10)
```

```
    total = len(top_mutations)
```

```
    for index, row in top_mutations.iterrows():
```

```
        # Update progress bar
```



```

percent_complete = int(((index + 1) / total) * 100)

my_bar.progress(percent_complete, text=f"Querying DNA Position:
{row['Position']}...")

# Hit the live API

disease, risk = fetch_disease_data(row['Chromosome'], row['Position'],
row['Original_DNA'], row['Mutated_DNA'])

results.append({
    "Chromosome": row['Chromosome'],
    "Mutation Position": row['Position'],
    "DNA Change": f"{row['Original_DNA']} → {row['Mutated_DNA']}",
    "Predicted Disease": disease,
    "Clinical Risk": risk
})

time.sleep(0.5) # Slight delay so we don't crash the free medical API

my_bar.empty() # Clear the loading bar when done

# Display the final Medical Report
st.success("✅ Clinical API Scan Complete.")

# Convert to a clean table
report_df = pd.DataFrame(results)

# Highlight high-risk diseases in the table
def highlight_risk(val):

```

```
color = '#ff4b4b' if 'pathogenic' in str(val).lower() else "  
return f'background-color: {color}'
```

```
st.dataframe(report_df.style.applymap(highlight_risk, subset=['Clinical Risk']),  
use_container_width=True)
```

```
except FileNotFoundError:
```

```
st.error("Spark data not found. Please run the Big Data engine first.")
```

Dataset Used:

Chromosome 22 sequencing data (FASTQ/BAM) sourced from the [1000 Genomes Project \(AWS Open Data\)](#) and the NIST [Genome in a Bottle \(GIAB\)](#) Consortium.